minicut 動画編集エディタ

組み込み JavaScript & プラグイン API リファレンスマニュアル
Duktape 2.7.0 エンジン 完全 UTF-8 / 日本語対応 ドッキング可能ツールパネル ツールシェルフ・ドック ネットワークライブラリ (libcurl)

1. はじめに & JavaScript アーキテクチャ

minicut は完全な **JavaScript** エンジン（Duktape 2.7.0）を内蔵しており、標準で完全な UTF-8 / Unicode 文字列操作をサポートしています。日本語の文字化けやバイト境界の問題がなく、国際化されたツールや自動化スクリプトをスムーズに開発できます。

スクリプトディレクトリの配置

ディレクトリ	役割 / 実行タイミング	概要
`~/.config/minicut/autorun/`	起動時自動登録	起動時に自動実行され、メニューバー下の「ドック (Dock)」にアイコンボタンを登録します（`.js` ファイル対応）。
`~/.config/minicut/scripts/`	メニュークリック時実行	メインメニューバーの **Scripts (スクリプト)** メニューに登録され、クリック時にパネルを開いて実行します。
`~/.config/minicut/icons/`	ドック用アイコン画像	スクリプト名と同名の 32x32 PNG アイコン画像（例: `text_animator.js` 用の `text_animator.png`）。

2. JavaScript プラグインの基本構造

```
// ================================================================================
// JavaScript プラグインの基本例: custom_animator.js
// ================================================================================

var targetTrack = 0;
var textContent = "日本語テキスト字幕";

function applyAnimation() {
    minicut.push_undo();
    var clipId = timeline.place_text_clip(targetTrack, textContent,
timeline.get_playhead(), 5.0);
    if (clipId) {
        clip.set_position(clipId, 0.0, 150.0);
        clip.set_scale(clipId, 1.2);
        timeline.add_keyframe(clipId, "opacity", 0.0, 0.0);
        timeline.add_keyframe(clipId, "opacity", 0.5, 1.0);
        minicut.set_status("字幕クリップを配置しました: " + textContent);
    }
}
```

```
plugin.init_panel = function() {
    gui.text("JavaScript テキストアニメーションツール");
    gui.separator();
    targetTrack = gui.slider_int("対象トラック", targetTrack, 0, 4);
    textContent = gui.input_text("字幕テキスト", textContent);

    if (gui.button("アニメーションテキストを配置", applyAnimation)) {
        // インライン処理
    }
};
```

3. コアモジュールリファレンス

3.1. プロジェクト & アセット操作 (`project` / Global)

関数	戻り値	説明
`project.save([path])` `project_save([path])`	`boolean`	プロジェクトを `.minicut` ファイルとして保存します。
`project.load(path)` `project_load(path)`	`boolean`	指定パスからプロジェクトを読み込みます。
`project.get_name()`	`string`	プロジェクト名を取得します。
`project.set_name(name)`	`void`	プロジェクト名を設定します。
`project.import_video(path)`	`string`	動画ファイルをアセットプールへインポートします。
`project.import_audio(path)`	`string`	音声ファイルをアセットプールへインポートします。
`project.import_image(path)`	`string`	静止画像ファイルをアセットプールへインポートします。
`assets.create_text(text, [opts])`	`string`	日本語や各言語のテキストアセットを生成します。

3.2. ネットワーク通信ライブラリ (`net`)

関数	戻り値	説明
`net.get(url)`	`object`	HTTP GET リクエスト。`{{status, body, ok, error}}` を返します。
`net.post(url, body)`	`object`	HTTP POST リクエスト（文字列または JSON）。

関数	戻り値	説明
net.download(url, dest)	boolean	ファイルを直接ローカルディスクへダウンロードします。
net.upload(url, path)	object	ローカルファイルをサーバへアップロードします。
net.request(options)	object	カスタム HTTP リクエスト。

3.3. タイムライン & トラック管理 (timeline)

関数	戻り値	説明
timeline.get_playhead()	number	現在の再生ヘッド位置（秒）を取得します。
timeline.set_playhead(t)	void	再生ヘッドを指定時刻へ移動します。
timeline.get_duration()	number	タイムラインの総再生時間を取得します。
timeline.get_track_count()	number	トラックの総数を取得します。
timeline.add_track(type, [name])	number	新規トラックを追加し、トラック番号を返します。
timeline.place_video(trk, id, start, dur)	string	動画クリップを配置し、クリップ ID を返します。
timeline.place_audio(trk, id, start, dur)	string	音声クリップを配置し、クリップ ID を返します。
timeline.place_image(trk, id, start, dur)	string	画像クリップを配置し、クリップ ID を返します。
timeline.place_text_clip(trk, text, start, dur)	string	テキストクリップを生成して配置します。
timeline.delete_clip(clip_id)	boolean	クリップとキーフレームを削除します。
timeline.split_clip(clip_id, [t])	boolean	指定時刻でクリップを分割します。

関数	戻り値	説明
`timeline.select_clips(ids_array)`	`void`	配列で指定した複数クリップを一括選択します。
`timeline.get_selected_clips()`	`Array<string>`	選択中のクリップ ID 配列を取得します。
`timeline.clear_selection()`	`void`	クリップ選択を全解除します。
`timeline.move_track_up(idx)`	`void`	トラックを 1 つ上に移動します。
`timeline.move_track_down(idx)`	`void`	トラックを 1 つ下に移動します。
`timeline.set_track_name(idx, name)`	`void`	トラック名を変更します。
`timeline.get_track_name(idx)`	`string`	トラック名を取得します。
`timeline.add_keyframe(id, prop, t, val)`	`void`	キーフレームアニメーションを追加します。
`timeline.remove_keyframe(id, prop, t)`	`void`	指定時刻のキーフレームを削除します。
`timeline.evaluate_keyframe(id, prop, t, [def])`	`number`	補間計算されたパラメータ値を取得します。

3.4. クリップ変形 & 属性操作 (`clip`)

関数	戻り値	説明
`clip.get_position(clip_id)`	`{{x, y}}`	中心座標オブジェクト `{{x: 0, y: 0}}` を返します。
`clip.set_position(clip_id, x, y)`	`void`	クリップの中心座標を設定します。
`clip.get_scale(clip_id)`	`number`	スケール倍率（1.0 = 100%）を取得します。
`clip.set_scale(clip_id, s)`	`void`	スケール倍率を設定します。
`clip.get_rotation(clip_id)`	`number`	回転角度（度数法）を取得し

関数	戻り値	説明
		ます。
`clip.set_rotation(clip_id, r)`	`void`	回転角度を設定します。
`clip.get_opacity(clip_id)`	`number`	不透明度 [0.0, 1.0] を取得します。
`clip.set_opacity(clip_id, o)`	`void`	不透明度を設定します。
`clip.get_volume(clip_id)`	`number`	音量倍率 [0.0, 2.0] を取得します。
`clip.set_volume(clip_id, v)`	`void`	音量倍率を設定します。
`clip.get_time(clip_id)`	`{{start_time, duration, offset}}`	タイミング情報を取得します。
`clip.set_time(clip_id, start, dur, [off])`	`void`	タイミングとオフセットを設定します。
`clip.get_info(clip_id)`	`object`	詳細プロパティオブジェクトを取得します。

3.5. GUI UI 要素 (gui)

関数	戻り値	説明
`gui.text(text)`	`void`	テキストを表示します。
`gui.text_colored(r, g, b, a, text)`	`void`	RGBA 色付きテキストを表示します。
`gui.button(label, [callback])`	`boolean`	ボタンを表示し、クリック時にコールバックを実行します。
`gui.slider_float(label, val, min, max, [cb])`	`number`	浮動小数点スライダーを表示します。
`gui.slider_int(label, val, min, max, [cb])`	`number`	整数スライダーを表示します。

関数	戻り値	説明
`gui.input_text(label, text, [cb])`	`string`	1 行テキスト入力欄を表示します。
`gui.checkbox(label, state, [cb])`	`boolean`	チェックボックスを表示します。
`gui.same_line()`	`void`	次のウィジェットを同一行に配置します。
`gui.separator()`	`void`	水平区切り線を描画します。
`gui.file_dialog(options)`	`void`	ファイルダイアログを開きます。

3.6. エンジンユーティリティ (`minicut` / `console`)

関数	戻り値	説明
`minicut.push_undo()`	`void`	Undo 用スナップショットを記録します。
`minicut.set_status(msg, [dur])`	`void`	トースト通知メッセージを表示します。
`minicut.log(...)` `console.log(...)`	`void`	ターミナルにデバッグログを出力します。